Typestate-oriented Programming with Ad-hoc Polymorphism on the Typestate

Oliver Tušla
tuslaoli@fit.cvut.cz
Czech Technical University
Prague, Czech Republic

1 Motivation

Requirements for programming language safety are ever-increasing. Some languages focus on preventing data races, memory safety, or type safety. Yet, those languages offer limited support for expressing a domain's real behaviour in the language itself and leveraging it to uphold the properties the domain necessitates.

Tracking proper initialization and ownership of objects can be done using existing techniques; however, there is limited support for checking a set of valid operations on an object at a point in the program's execution in a unified way.

Consider a pipeline program. A pipeline needs to collect tasks along with their dependencies. Once the tasks are collected, it must ensure there are no circular dependencies; if not, the pipeline can be executed. Failing to check this as a user of the pipeline can be fatal. It is desired that a program using the pipeline incorrectly is not constructible in the first place.

2 Background

Typestate[4][7] is a programming concept that incorporates the state of an object into its type. This information is subsequently used to decide which operations on the object are valid given its typestate.

Research has focused on applying typestate analysis as an additional layer[5][3][8] and integrating it into a calculus[6] or the core of programming languages like Plaid[4][1] and Obsidian[2].

3 Approach

Consider a pipeline program from Figure 1. The acyclic property is easily tracked using `clauses`. A newly constructed pipeline is empty and thus does not contain any cycles. We then track changes to this property throughout the life cycle of a particular instance. Constructing a fresh task in a pipeline does not invalidate previous knowledge about dependencies since there is no task with a dependency on the new one. Adding an arbitrary dependency between two tasks already existing in a pipeline may or may not create a cycle. Thus, in the `add_dependency` method, we declare that by calling this method, we `make` the instance into one with unknown or any cyclic state.

Subsequently, if we want to execute the pipeline, like in Figure 2, we first need to ensure the pipeline does not contain any cycles as mandated by the `execute` method. In the `process` function, we recover a lost state property using the `decide` statement and handle both an executable and non-executable pipeline.

Since our program already tracks the state, we can also leverage this information to provide different method implementations based on it. Consider the task class from Figure 3. The method `depend_on` is specialized for each state we care about. Along with the specialized implementations, we also provide a general implementation which works for any state. Such a version might not exist for some problems, and we will be notified by the checker.

4 Contributions

There is an increasing need for safety in modern programs. Newly developed languages pay high attention to mutability tracking, aliasing protection, and ownership. We concentrate on expressing the domain properties of a model using the program itself and providing facilities to use the model safely.

- We provide typestate checking using a concept of three-valued clauses which express properties of a class in time.
- We present a system for dissecting program behaviour into chunks of code that are easy to reason about using ad-hoc polymorphism on typestate and the guarantees of clauses.
- We show how the system can be leveraged to write succinct programs and handle situations where the checker cannot verify the program using decider statements.

```
class Pipeline:
  clause Acyclic
  var tasks: [Task]

  static method new() -> Pipeline makes Acyclic =
    Pipeline: tasks = []

 method add_dependency(from: Id, to: Id) makes any Acyclic =
    var t1 = tasks[from]
    t1.depend_on(to) // any t1.Acyclic

 method add_task(name: String) -> Id =
    tasks = tasks + [Task(name)]
    return tasks.last()

 method add_task_with_dep(name: String, dep: Id) =
    let id = add_task(name)
    // we are allowed to overwrite the makes
    // because we are the domain designers
    add_dependency(id, dep)

 method check_acyclic() -> CertainClauseVal<Acyclic> =
    // try to find cycles, Acyclic::Is or Acyclic::Not

 method execute() takes Acyclic =
    // topsort and run in order
```

Figure 1: Pipeline class

```
fn pipeline_1() -> Pipeline makes any Acyclic =
  let pipeline = Pipeline::new() // pipeline.Acyclic
  let code_id = pipeline.add_task("Code")
  let qa_id = pipeline // pipeline.Acyclic
    .add_task_with_dep("QA", code_id)
  let cr_id = pipeline // pipeline.Acyclic
    .add_task_with_dep("Code Review", code_id)
  let depl_id = pipeline // pipeline.Acyclic
    .add_task_with_dep("Deploy", qa_id)
  pipeline.add_dependency(depl_id, cr_id)
  return pipeline // any pipeline.Acyclic

fn pipeline_2() -> Pipeline makes any Acyclic =
  let pipeline = Pipeline::new() // pipeline.Acyclic
  let eat_id = pipeline.add_task("Eat")
  let sleep_id = // pipeline.Acyclic
    pipeline.add_task_with_dep("Sleep", eat_id)
  pipeline.add_dependency(sleep_id, eat_id)
  return pipeline // any pipeline.Acyclic

fn process() =
  let pipeline = pipeline_X() // any pipeline.Acyclic
  decide pipeline Acyclic by pipeline.check_acyclic:
    // we will reach this case for pipeline_1()
    pipeline.execute()
  otherwise:
    // we will reach this case for pipeline_2()
    println("We can't execute this, sorry. :-/")
```

Figure 2: Sample usage

```
class Task:
  clause Dependent
  let name: String
  var deps: Option<[Task]>

  static method new(name: String) -> Task
  makes not Dependent, deps.None =
    Task: name = name, deps = None

  method depend_on(id: Id) =
    match deps:
      Some(tasks) => deps = Some(tasks + [id])
      None => deps = Some([id])

  method depend_on(id: Id)
  takes not Dependent, deps.None
  makes Dependent, deps.Some =
    deps = Some([id])

  method depend_on(id: Id)
  takes Dependent, deps.Some
  makes Dependent, deps.Some =
    deps = Some(deps.get() + [id])

 method check_dependent() -> CertainClauseVal<Dependent> =
    if deps.is_some(): CertainClauseVal::Is
    else: CertainClauseVal::Not
```

Figure 3: Task class

References

[1] Jonathan Aldrich, Joshua Sunshine, Darpan Saini, and Zachary Sparks. 2009. Typestate-oriented programming. In *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications* (Orlando, Florida, USA) *(OOPSLA '09)*. Association for Computing Machinery, New York, NY, USA, 1015–1022. doi:10.1145/1639950.1640073

[2] Michael Coblenz, Reed Oei, Tyler Etzel, Paulette Koronkevich, Miles Baker, Yannick Bloem, Brad A. Myers, Joshua Sunshine, and Jonathan Aldrich. 2020. Obsidian: Typestate and Assets for Safer Blockchain Programming. *ACM Trans. Program. Lang. Syst.* 42, 3, Article 14 (Nov. 2020), 82 pages. doi:10.1145/3417516

[3] Robert DeLine and Manuel Fähndrich. 2004. Typestates for Objects. In *ECOOP 2004 – Object-Oriented Programming*, Martin Odersky (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 465–490.

[4] Ronald Garcia, Éric Tanter, Roger Wolff, and Jonathan Aldrich. 2014. Foundations of Typestate-Oriented Programming. *ACM Trans. Program. Lang. Syst.* 36, 4, Article 12 (Oct. 2014), 44 pages. doi:10.1145/2629609

[5] João Mota, Marco Giunti, and António Ravara. 2021. Java Typestate Checker. In *Coordination Models and Languages: 23rd IFIP WG 6.1 International Conference, COORDINATION 2021, Held as Part of the 16th International Federated Conference on Distributed Computing Techniques, DisCoTec 2021, Valletta, Malta, June 14–18, 2021, Proceedings* (Valletta, Malta). Springer-Verlag, Berlin, Heidelberg, 121–133. doi:10.1007/978-3-030-78142-2_8

[6] Hannes Saffrich, Yuki Nishida, and Peter Thiemann. 2024. Law and Order for Typestate with Borrowing. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 323 (Oct. 2024), 29 pages. doi:10.1145/3689763

[7] Robert E. Strom and Shaula Yemini. 1986. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929

[8] André Trindade, João Mota, and António Ravara. 2020. Typestates to Automata and back: a tool. *Electronic Proceedings in Theoretical Computer Science* 324 (Sept. 2020), 25–42. doi:10.4204/eptcs.324.4